

[TODO: title – current favourite: Communication-Aware DSE for IMC DNN Accelerators: Co-Optimizing Crossbar Packing, Tile Placement, and Adaptive 3D-NoC Selection]

[TODO: authors]
[TODO: affiliation]

Abstract—[TODO: abstract – write last]

I. INTRODUCTION

Deep Neural Networks (DNNs) have become the dominant workload class across computing. They underpin the vast majority of AI-based applications, and DNN inference has become infrastructure, running continuously from edge devices to warehouse-scale data centers [1]. The latency and energy budget of DNN accelerators is therefore a first-order design metric, and a primary axis of competition among chip designers and manufacturers. Weight movement is what drives this budget in conventional accelerators [2], [3]. In-Memory Computing (IMC) eliminates weight traffic by keeping weights in place and performing Multiply-Accumulate (MAC) operations locally. This alleviates the weight-movement bottleneck, improving performance while reducing the communication energy budget and launching a new generation of IMC DNN accelerators [4], [5], [6]. However, single array capacity is nowhere near occupying all weights for a full, DNN making tile architecture inevitable. For example one ResNet-50 block requires 736 crossbars (CBs) and a full network requires over ten thousand. Because ADC peripherals have a large area and power overhead, they are shared across the area of a tile. This limits the number of CBs that fit in a tile resulting in the computation being split among hundreds or even thousands of tiles connected through Network-on-Chip (NoC) [5].

The consequence is that IMC does not eliminate communication; it relocates it. What remains mobile is activation scatter traffic and the dominant partial-sum reduction traffic: with default packaging, roughly 82% of a block’s NoC bytes is reduction that converge on few accumulator tiles via the on-chip network. The accelerator’s bottleneck that governs its performance and energy becomes a communication, and the design focus shifts from being computation-centric to communication-centric.

This paper studies ...

II. BACKGROUND AND RELATED WORK

Wafer-scale integration shows what collapsing memory into compute costs at the largest scale yet built: Cerebras’ WSE-3 holds 44 GB on-wafer at 21 PB/s while provisioning 214 Pb/s

of on-chip fabric — as reported, more aggregate bandwidth in the network than in the memory it serves [7]. Crossbar-based accelerators make the same collapse at tile granularity, so the interconnect rather than memory becomes the resource that mapping must be designed against [7].

III. SYSTEM MODEL AND METRICS

[TODO: fold the definitions block in results back here if space allows]

IV. METHODOLOGY

[TODO: converter, offline scorer, search, simulator setup]

V. RESULTS

A. Metrics and operating point

For a packing (c, r, s) and a placement of its tiles onto the $6 \times 6 \times 3$ mesh, all predictors below are computed offline from the traffic table alone.

- **Port floor** $PF = \max(PIL, PEL)$: the peak per-node injection (PIL) or ejection (PEL) rate, in flits/cycle. Both terms are fixed by the flow graph, so PF is *placement-invariant*: remapping relabels which node is hot, never how hot it is (verified identical to six decimal places across random permutations).
- **Peak link load** PL: the maximum offered rate on any router-to-router channel when each flow spreads evenly over its admissible minimal paths under the routing function. PL is the only term a placement moves.
- **Binding load** $BIND = \max(PF, PL)$.
- **Escapable fraction** $E = 1 - PL_f/PL$ at the peak link, where PL_f counts only flows for which *every* admissible path crosses that link. E is the share of the hot link’s load that a selection policy could, in principle, divert.
- **Per-mapping capacity** k^* : the injection scale at which a mapping reaches twice its own free-flow delay, interpolated from a load ladder. Mappings are compared at their own k^* , never at one global load [TODO: cite the k^* method note].

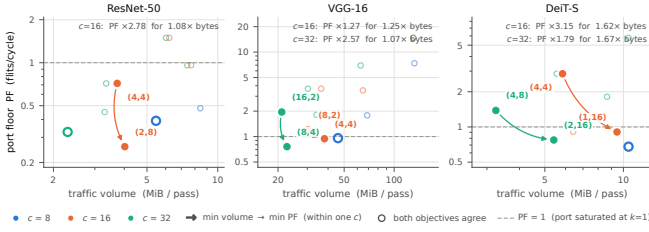


Fig. 1. The packing design space: traffic volume against port floor PF, all feasible (c, r, s) points, argmins taken *within* each density c . Arrows run from the minimum-volume orientation to the minimum-PF orientation at the same c ; rings mark cells where the two objectives agree. This figure argues about the floor; that the floor determines delay is established in Fig. 2.

Load is reported as the fraction $PF \cdot k$ of the binding port's capacity; the three workloads all knee at 44–48% of the dominant reduction sink's ejection bandwidth, so this normalization is workload-independent.

B. Packing sets the floor; and minimum traffic does not minimize it

At fixed density c , the orientation (r, s) decides how much of the partial-sum reduction is internalised on-tile and how much reaches the NoC, and thereby sets PF: across ResNet-50's eleven feasible (c, r, s) points PF spans 0.258–1.494 flits/cycle (5.8 $\times$) before a single placement decision is made. Placement, by contrast, moves only PL (span 7.5 $\times$ within one packing). The two terms of BIND therefore belong to different design layers — PF to packing, PL to placement — and the rest of this section establishes which layer holds control in which regime.

Fig. 1 plots every feasible packing of the three workloads in the (traffic volume, PF) plane. The classical pre-NoC objective — minimize communication volume — and the architecture-aware objective — minimize PF — select *different* orientations in five of the eight (*workload, c*) cells where more than one orientation exists. The disagreement is not marginal: at $c=16$, ResNet-50's minimum-volume orientation (4, 4) carries a 2.78 $\times$ higher floor than (2, 8), which costs only 8% more traffic; VGG-16 at $c=32$ shows the same pattern (2.57 $\times$ the floor, 7% traffic). Where the trade is expensive the figure says so: DeiT-S buys its 1.8–3.2 $\times$ floor reductions with 1.6 $\times$ the traffic. Minimum bytes selects the busiest packings; it is the wrong objective whenever the floor, not the volume, binds — which Sec. V-C shows is exactly the regime that matters.

C. The regime boundary: delay is flat below the floor and steep above

Fig. 2 plots mean delay against PL for 24 placements of one packing spanning $PL = 0.127$ – 0.490 (3.9 $\times$) at two loads, with path variety pinned so PL is the only moving predictor. Delay does not respond to PL until PL crosses PF, and responds sharply after: a hinge regression at the floor yields $R^2 = 0.735$ (vs. 0.404 linear), with a below-floor slope indistinguishable from zero ($t = -0.16$) and an above-floor slope of +655 cycles per unit load ($t = +5.13$, $p < 10^{-4}$). The hinge

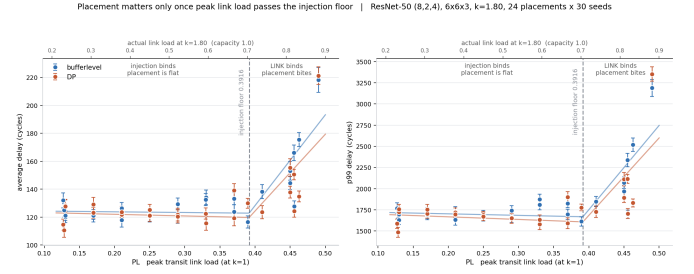


Fig. 2. Mean delay against peak link load PL under local (buffer-level) selection, PF marked. Below the floor a 3.07 $\times$ spread in PL moves delay 1.15–1.18 $\times$ (n.s.); above it a 1.20 $\times$ spread moves delay 1.71–1.85 $\times$. [TODO: replace with E1-grid version, BL only]

replicates at a second load level ($R^2 = 0.705$; slope ratio 64–72 $\times$ across k) [TODO: regenerate fig with the E1 grid; cite exact fig file].

Below the floor, the packing orientation is the determining layer. With $BIND = PF$, delay tracks the floor itself [TODO: E5: PF varied through (r, s) at matched k — currently varied through k only; run E5 before submission], and placement is inert: seventeen placements spanning 3.07 $\times$ in PL move mean delay 1.15 $\times$ (n.s.). We stress the constructive reading: *verifying* $PL < PF$ *freed the entire mapping budget* for objectives that are not latency — thermal spreading, energy, load balance — at no delay or p99 penalty. On DeiT-S this licence replicates end-to-end: at two packings held below their floors, a placement searched for escapable fraction plus a free choice of selection policy buys 0.99–1.11 $\times$ over the classical minimum-hop mapping with local selection (1,344 runs; indistinguishable from parity at most loads).

The licence carries one condition: *margin*. Among eight placements all below the floor, capacity k^* still varies 1.28 $\times$ and correlates with proximity to the boundary ($r(PL/PF, k^*) = -0.70$); at mid-load the two placements nearest the floor degrade 13.6 $\times$ earlier than the rest. The budget is free at $PL \lesssim 0.6 PF$, not asymptotically up to the boundary.

Above the floor, control passes to placement. Across the 24-placement grid of Sec. V-E, PL predicts per-mapping capacity at $r(PL, k^*) = -0.60$ ($p = 0.002$), and capacity spans 1.50 $\times$ across placements of the *same* packing. This is also the first simulator validation of the offline PL model ($R^2 = 0.37$): a moderate capacity predictor, no more [TODO: VGG replication: $r(PL, k^*) = -0.72$, BL arm; DP arm and DeiT-S pending (E7)].

D. Selection pays only above the floor, and only where load can escape

Table I compares the distributed cost-to-go policy (DP) against the local buffer-level heuristic (BL) on identical placements, split by regime. Below the floor the policies are statistically indistinguishable — no routing decision reaches a flit that has not entered the network — while above it DP removes 7–10% of mean delay. The gain is not universal: it requires escapable load. Pooled across five workload/packing arms

TABLE I
DP VS. BL MEAN DELAY ON IDENTICAL PLACEMENTS, BY REGIME (24
PLACEMENTS, $n=30$ SEEDS).

regime	$k = 1.60$		$k = 1.80$	
	Δ	p	Δ	p
below floor ($n=17$)	-0.0%	0.98	-1.1%	0.37
above floor ($n=7$)	-10.0%	0.0005	-6.7%	0.008

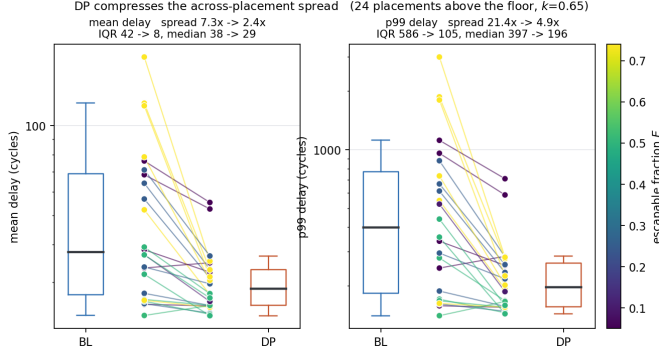


Fig. 3. Delay across placements of equal peak link load, BL vs. DP, above the floor ($k = 0.65$). DP compresses the spread across placements rather than shifting its floor. [TODO: final styling pass; confirm k in caption]

($n=71$ placements), DP’s capacity gain tracks the escapable fraction E on ejection-bound packings ($r = +0.59$, $p = 0.0002$) and vanishes on injection-bound ones ($r = +0.07$, n.s.), where the binding port is upstream of every routing decision.

E. Adaptive selection returns the mapping freedom that congestion took

The headline effect of DP is not its mean — it is its *variance across placements*. Fig. 3 compares the two policies over placements of equal PL above the floor. DP does not make the best placements faster (BL matches or beats it there); it removes the tail: across 21 equal-PL placements the delay spread contracts from $1.59\times$ to $1.45\times$ ($CV\ 0.145 \rightarrow 0.101$), and in a population containing BL-collapse cases from $64\times$ to $8.6\times$. On the capacity axis the same grid gives DP a mean $+10.3\%$ k^* gain ($p = 2.3 \times 10^{-4}$, positive on 18/24), sized by E : a two-predictor model gain $\sim E + PL$ reaches adjusted $R^2 = 0.55$, and every placement where DP loses is low- E .

The design consequence mirrors Sec. V-C: once a PL level is fixed, the choice *among* equal-PL placements matters far less under DP than under BL — adaptive selection extends the below-floor mapping licence into the congested regime. What it does not do is substitute for the placement decision itself: the k^* spread across PL levels survives under both policies.

F. Scope and threats to validity

PL, PL_f and E are offline path-enumeration models; only PL has been validated against the simulator, and only as a moderate capacity predictor ($R^2 = 0.37$). The even-spreading assumption behind PL models an ideal balancer, which BL is

not — BL underperforms its own PL estimate in proportion to E . All results use one $6\times 6\times 3$ mesh; mesh-size generality is out of scope. Partial sums are carried at INT32, a convention that *overstates* the reduction traffic the contribution exploits; the placement rankings survive at 16-bit for ResNet and VGG. Cross-workload replication is complete for the hinge and the licence, partial for the selection results [TODO: close with E7 + VGG DP arm, then finalize].

VI. CONCLUSION

[TODO: conclusion + the three directions (temporal DP, thermal-aware mapping, router design for reduction sinks)]

REFERENCES

- [1] e. a. Jouppi, “In-datacenter performance analysis of a tensor processing unit,” vol. 45, no. 2. New York, NY, USA: Association for Computing Machinery, Jun. 2017, p. 1–12. [Online]. Available: <https://doi.org/10.1145/3140659.3080246>
- [2] M. Horowitz, “1.1 computing’s energy problem (and what we can do about it),” in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014, pp. 10–14.
- [3] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [4] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, “Memory devices and applications for in-memory computing,” *Nature nanotechnology*, vol. 15, no. 7, pp. 529–544, 2020.
- [5] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, “Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars,” in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 14–26.
- [6] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, “Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory,” in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 27–39.
- [7] T. Miyajima and L. Fukuoka, “Benchmarking the cerebrar wafer scale engine-2 architecture,” ser. SC Workshops ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 818–822. [Online]. Available: <https://doi.org/10.1145/3731599.3767438>